

Multimodal Handwritten Notes Reconstruction Tool

Final Product & Technical Specification

Purpose. A React + FastAPI application accepts 20–50 lecture/class photos, temporarily analyzes them, reconstructs their hierarchy and visual content, generates a handwritten/notebook-style PDF, permanently stores the PDF with an appropriate title, and deletes source photos after processing.

Core principle: source photos are ephemeral input; structured notes are the processing contract; the PDF is the persistent artifact.

Stack: React + TypeScript + Tailwind CSS; Python + FastAPI + Pydantic; PostgreSQL; Redis + worker queue; temporary storage; vision-capable AI; ReportLab/SVG rendering; private object storage for PDFs.

1. Functional Requirements

- Accept 20–50 images per job.
- Preserve original image sequence.
- Use the repeated main title on each image to group related content.
- Preserve subtitles/subtopics under the correct main title.
- Preserve paragraphs, bullets, numbered lists, formulas, tables, diagrams, photographs, graphs, and captions.
- Never flatten the hierarchy into one list.
- Do not invent missing content.
- Generate a handwritten/notebook-style PDF.
- Main title: Times New Roman, 20 pt, bold.
- Subtitle: Times New Roman, 16 pt, bold.
- Regular text and bullets: Times New Roman, 12 pt.
- Visuals must be preserved or reconstructed appropriately.
- Store the final PDF permanently; never permanently store source photos.
- Delete temporary photos after success, failure, or abandonment cleanup.
- Generate an appropriate PDF title/filename.

Example hierarchy

```
CHAPTER 4 20 pt bold

Definition 16 pt bold
• Point 12 pt
• Point 12 pt

Example 16 pt bold
Explanation 12 pt

Benefits 16 pt bold
• Benefit 1 12 pt
• Benefit 2 12 pt
```

2. Image & Visual Content Handling

Do not treat every visual region as OCR text. Classify visual regions into text, formula, table, diagram, graph, photograph/image, and caption.

```
Source Photo
|
+-- Text -----> structured text
+-- Formula -----> math representation
+-- Table -----> structured table
+-- Diagram -----> reconstruct or preserve
+-- Graph -----> reconstruct or preserve
+-- Photograph ---> preserve/crop
+-- Caption -----> associate with visual
```

For semiconductor structures, circuit diagrams, flowcharts, graphs, anatomy figures, and block diagrams, reconstruct simple/high-confidence diagrams as SVG/vector content. For complex or technically sensitive visuals, preserve the original cropped visual rather than risk changing technical meaning.

Accuracy rule: technical correctness is more important than visual imitation. A preserved diagram is preferable to a beautiful but incorrect redraw.

3. High-Level Architecture

```
React / TypeScript
|
v
FastAPI API
| | |
v v v
PostgreSQL Temp Redis/Queue
metadata images |
Worker
|
+-----+-----+
| |
v v
Image preprocessing Vision AI
|
v
Structured extraction JSON
|
v
Reconstruction engine
|
+-----+-----+
| | |
v v v
Text Formulas Visuals
+-----+-----+
|
v
PDF renderer
|
v
Permanent PDF storage
|
v
React download/library

Source photos -> DELETE after processing
```

Keep AI extraction, reconstruction, PDF rendering, and storage independent. This lets the same structured notes produce multiple output styles later.

4. Privacy & Storage Model

Permanent: PDF, PDF metadata, job status, timestamps, user ownership, and storage reference.

Temporary: uploaded photos, processed copies, image crops, and intermediate extraction data.

- Never store source image blobs in PostgreSQL.
- Never put source photos in the permanent PDF library.
- Delete source images in a finally/cleanup path.
- Run periodic cleanup for abandoned jobs.
- Use private object storage for PDFs.
- Use authenticated access or short-lived signed URLs.
- Use UUID job identifiers.
- Review third-party AI retention/training policies separately from your own storage policy.

```
Temporary:
/tmp/note-jobs/{job_id}/001.jpg ... 050.jpg
```

```
Permanent:
```

```
notes/{year}/{month}/{job_id}/{sanitized-title}.pdf
```

5. Processing State Machine

```
CREATED -> UPLOADING -> UPLOADED -> PREPROCESSING  
-> ANALYZING -> RECONSTRUCTING -> RENDERING  
-> GENERATING_PDF -> COMPLETED
```

```
Any processing state -> FAILED  
Expired/abandoned temporary job -> CLEANED
```

The job must not become COMPLETED until the final PDF is successfully persisted.

6. AI Extraction Contract

The vision model extracts structured content. It does not decide final typography or generate the final PDF.

```
{
  "image_number": 7,
  "main_title": "Chapter 4",
  "elements": [
    {
      "type": "subtitle",
      "text": "Benefits",
      "children": [
        {"type": "bullet", "text": "Helps quantify uncertainty."}
      ]
    },
    {
      "type": "diagram",
      "image_id": "visual_7_1",
      "confidence": 0.94
    }
  ]
}
```

- Identify the main title on every image.
- Identify subtitles separately from regular text.
- Preserve bullets and numbered lists.
- Preserve original element order.
- Extract formulas accurately.
- Identify tables and row/column structure.
- Identify diagrams, graphs, and photographs.
- Associate captions with the correct visual.
- Do not silently invent unreadable content.

7. Main Title Grouping

```
Image 1 -> Chapter 4
Image 2 -> Chapter 4
Image 3 -> Chapter 4
Image 4 -> Chapter 5

Result:
Chapter 4
+ image 1 content
+ image 2 content
+ image 3 content

Chapter 5
+ image 4 content
```

Normalize capitalization, trailing punctuation, and whitespace. Avoid aggressive fuzzy matching: Chapter 4 and Chapter 5 must remain separate. If a title appears again later after another title, preserve source order rather than automatically merging distant occurrences.

8. Structured Document Model

```
Document
|
+-- Section
+-- main_title
```

```

+-- Element
+-- subtitle
+-- paragraph
+-- bullet
+-- numbered
+-- formula
+-- image
+-- diagram
+-- graph
+-- table
+-- caption

{
  "sections": [{
    "main_title": "Chapter 4",
    "elements": [
      {
        "type": "subtitle",
        "text": "Definition",
        "children": [
          {"type": "bullet", "text": "Point one"},
          {"type": "bullet", "text": "Point two"}
        ]
      }
    ]
  }]
}

```

This tree is the contract between AI extraction and deterministic PDF rendering.

9. PDF Rendering Specification

Element	Font	Size	Weight
Main title	Times New Roman	20 pt	Bold
Subtitle	Times New Roman	16 pt	Bold
Regular text	Times New Roman	12 pt	Normal
Bullet text	Times New Roman	12 pt	Normal
Formula	Math-compatible rendering	~12 pt	Normal
Caption	Times New Roman	10–12 pt	Normal

If Times New Roman is mandatory, 'handwritten mode' should be implemented as a notebook/layout aesthetic: paper background, underlines, boxes, arrows, highlighting, controlled spacing, and restrained visual variation. An actual handwriting font should be a separate style because it conflicts with the Times New Roman requirement.

For simple diagrams, SVG is recommended as an intermediate representation. SVG maintains sharp lines and permits controlled handwritten-style transformations.

10. FastAPI Backend

```
backend/  
app/  
main.py  
api/  
notes.py  
downloads.py  
models/  
note.py  
schemas/  
note.py  
document.py  
services/  
image_service.py  
vision_service.py  
reconstruction_service.py  
pdf_service.py  
visual_service.py  
storage_service.py  
workers/  
note_worker.py  
utils/  
cleanup.py  
filename.py  
config.py  
fonts/  
tests/  
requirements.txt  
Dockerfile
```

Core endpoints

```
POST /api/v1/notes  
Upload 20–50 images and create a job.  
  
GET /api/v1/notes/{job_id}  
Return status, progress, title, and metadata.  
  
GET /api/v1/notes/{job_id}/download
```

Download/stream the permanent PDF.

```
DELETE /api/v1/notes/{job_id}
```

Optional: delete the saved PDF and metadata.

Do not keep the HTTP request open while processing 50 images. Create the job, save temporary inputs, enqueue processing, and return the job ID.

11. PostgreSQL Model

```
CREATE TABLE notes (  
  id UUID PRIMARY KEY,  
  user_id UUID,  
  title VARCHAR(255) NOT NULL,  
  status VARCHAR(50) NOT NULL,  
  image_count INTEGER NOT NULL,  
  page_count INTEGER,  
  style VARCHAR(50),  
  pdf_storage_key TEXT,  
  created_at TIMESTAMP NOT NULL,  
  completed_at TIMESTAMP,  
  error_message TEXT  
);
```

The database contains metadata, not image blobs.

12. React Frontend

```
frontend/  
src/  
components/  
ImageUploader.tsx  
ImagePreview.tsx  
ProcessingProgress.tsx  
PdfResult.tsx  
PdfLibrary.tsx  
pages/  
Home.tsx  
NoteDetail.tsx  
services/  
api.ts  
types/  
note.ts
```

User flow

1. Select 20-50 images
2. Preview thumbnails
3. Reorder if needed
4. Choose output style
5. Generate Notes
6. Show processing progress
7. Show generated title
8. Preview/download PDF
9. Saved PDFs appear in library

The frontend never controls privacy cleanup; backend/storage lifecycle logic is authoritative.

13. Visual Reconstruction Strategy

Simple/high-confidence diagram:

```
Photo -> detect diagram -> identify shapes/labels/arrows  
-> structured diagram -> SVG  
-> handwritten visual treatment -> PDF
```

Complex/technical diagram:

```
Photo -> detect/crop original diagram  
-> preserve original visual  
-> notebook presentation/caption -> PDF
```

- Semiconductor / P-N junction diagrams.
- Circuit diagrams.
- Graphs and coordinate plots.
- Flowcharts.
- Scientific/anatomical figures.
- Block diagrams.
- Photographs embedded in lecture material.
- Tables and comparison charts.

The system should favor technical correctness over visual imitation.

14. PDF Naming & Permanent Storage

```
Detected title:  
Chapter 4 - Probability
```

Permanent filename:
Chapter_4_-_Probability.pdf

Storage:
notes/2026/09/{job_id}/Chapter_4_-_Probability.pdf

The user may rename the final PDF later without affecting the source-photo lifecycle.

15. Security

- Authenticate users if PDFs are private.
- Authorize every PDF download against the owning user.
- Use UUIDs rather than predictable IDs.
- Validate MIME type, extension, file size, and aggregate upload size.
- Sanitize generated filenames.
- Use private object-storage buckets.
- Use short-lived signed URLs.
- Do not log raw images unnecessarily.
- Rate-limit job creation.
- Limit concurrent processing per user.
- Delete temporary files on success and failure.
- Monitor third-party AI data handling.

16. Error Handling & Cleanup

```
def process_job(job_id):
    try:
        update_status(job_id, "PREPROCESSING")
        preprocess_images(job_id)

        update_status(job_id, "ANALYZING")
        extracted = analyze_images(job_id)

        update_status(job_id, "RECONSTRUCTING")
        document = reconstruct(extracted)
        validate_document(document)

        update_status(job_id, "RENDERING")
        pdf = render_pdf(document)

        storage_key = persist_pdf(pdf)
        mark_completed(job_id, storage_key)

    except Exception as exc:
        mark_failed(job_id, str(exc))

    finally:
        delete_temporary_job_files(job_id)
```

A scheduled cleanup worker is still required for abandoned jobs where the worker process crashes before finally executes.

17. Performance & Scalability

- Normalize high-resolution phone photos before vision processing.
- Use asynchronous workers for AI/PDF generation.
- Limit AI concurrency.
- Limit jobs per user.
- Track processing duration and AI cost.
- Use bounded retries with exponential backoff.
- Keep temporary storage isolated per job.
- Do not add Kafka/Kubernetes/microservices until workload justifies them.

18. Testing Strategy

- 20-image and 50-image jobs.
- Repeated title across pages.
- Different title between chapters.
- Same title appearing later after another chapter.
- Multiple subtitles under one title.
- Bullets continuing across images.
- Formulas and mathematical notation.
- Tables.
- Semiconductor/circuit/scientific diagrams.
- Photographs.
- Unreadable/low-quality text.
- Title capitalization differences.
- AI timeout/retry.
- Worker crash.
- PDF storage failure.
- Cleanup after success.
- Cleanup after failure.
- Cleanup after abandonment.

Critical successful-job invariant:

```
temporary source-image directory == absent  
AND permanent PDF == exists  
AND database status == COMPLETED  
AND database contains no source-image blobs
```

19. Implementation Roadmap

Phase 1 — Upload & privacy boundary

- FastAPI upload endpoint
- 20–50 validation
- temporary job directories
- PostgreSQL job model
- cleanup utility

Phase 2 — AI extraction

- Vision integration
- strict JSON schema
- main-title extraction
- subtitle extraction
- visual classification

Phase 3 — Reconstruction

- Title normalization
- same-title grouping
- hierarchy preservation
- visual placement
- formula/table handling

Phase 4 — PDF engine

- Times New Roman typography
- 20/16/12 hierarchy
- notebook layout
- diagram/image rendering
- appropriate filenames

Phase 5 — React product

- Drag-and-drop uploader
- thumbnail preview/reordering
- progress screen
- PDF preview/download
- saved PDF library

Phase 6 — Production hardening

- Redis + worker queue
- private object storage
- signed downloads
- rate limits
- monitoring
- scheduled cleanup

- retry policies

20. Final Product Contract

- Accept 20–50 photos.
- Analyze each source image for the job.
- Group repeated main titles.
- Preserve every subtopic.
- Render main titles at 20 pt bold.
- Render subtitles at 16 pt bold.
- Render regular text at 12 pt Times New Roman.
- Handle diagrams, graphs, tables, photographs, and formulas as distinct content types.
- Reconstruct simple diagrams as handwritten-style vector graphics.
- Preserve complex/technically sensitive visuals when reconstruction could introduce errors.
- Keep visuals in logical position.
- Store the final PDF permanently.
- Delete source photos after processing.
- Keep AI extraction and rendering as separate layers.
- Use asynchronous processing in production.
- Protect PDF downloads.

21. Final Engineering Principle

Do not build this as OCR → PDF. Build it as Image Understanding → Structured Document Reconstruction → Visual Reconstruction → Deterministic PDF Rendering.

This separation is what lets the system preserve titles, subtitles, bullets, formulas, diagrams, photographs, and technical visuals while producing a consistent handwritten-style document.